PROPRIETARY
INFORMATION

GRAPHITI API LIBRARY

Setup guide

Version 1.1

20th March 2025

Copyright © 2026 Orbit Research

Revision History

Rev.	Date	Description of Changes	Author
1.0	24-Nov-2025	First Version	Himanshu Makwana
1.1	20 Mar 2026	Update	Himanshu Makwana

Table of Contents

1 Introduction **3**
2 Purpose and Definition of Work **3**
3 Setting up device **4**
4 Prerequisite Software **4**
5 Setting up your coding environment **9**
6 Writing a CPP project **13**

1 Introduction

The Graphiti API library is a software development toolkit designed to interact with the Graphiti and Graphiti Plus devices by Orbit Research. The library provides developers with a comprehensive set of functions to access and control the devices, enabling communication through Serial, HID, and Draw modes. With support for multiple programming languages including C++, Python, and Java, the Graphiti API facilitates the development of custom applications that leverage the devices' unique input and output capabilities.

This document serves as a setup guide and reference for integrating the Graphiti API into development environments, providing instructions for installing prerequisites, configuring the coding environment, and writing applications using the API.

2 Purpose and Definition of Work

2.1 Purpose

The primary purpose of this work is to provide developers with a clear and structured approach to integrating the Graphiti and Graphiti Plus devices into software applications. The Graphiti API library abstracts the complexities of device communication, allowing developers to focus on building functional and interactive applications without worrying about low-level device handling.

2.2 Definition of the Work

This project involves setting up the Graphiti and Graphiti Plus devices, configuring the necessary software development environment, and utilizing the Graphiti API to build applications. The work covers:

1. Installation of prerequisite software, including Git, Python, Java JDK, Visual C++ Build Tools, and CMake.
2. Setting up the development environment with Visual Studio Code and configuring compiler and generator tools.
3. Using the Graphiti API to communicate with the device, handle events, and perform actions such as drawing, key input, and touch input.
4. Providing wrapper support for C++, Python, and Java, enabling developers to use the API in their preferred programming language.

3 Setting up device

3.1 Setting up Graphiti device

1. To start, connect your device to the Graphiti using a USB-A to Micro USB cable by connecting the USB-A to your device and the Micro USB cable to the Graphiti located on the left side of the Graphiti closest to Dot 3.
2. Next power the device on and press Space + Dot 8 or Space + Dot 7 to put the device into API mode. The key command Space + Dot 8 is for the Serial mode, and Key command Space + Dot 7 is for the HID mode. Alternatively, one can press Space + Dot 1 to put the device into Draw mode to enable the Draw API which includes Draw events.
3. Open the device manager from the Windows search bar in the taskbar by searching device manager
4. If you have selected the Serial mode on the device, then identify the COM port of the device by checking the Ports under “Ports (COM & LPT)” in the Device Manager Window. Identify the COM port the device has connected to, which is “COM” followed by

   a number, e.g., COM4, and note this for later

4 Prerequisite Software

4.1 Install git

1. Go to the official Git website: https://git-scm.com/
2. Click the “Download for Windows” button. This will download an .exe installer (e.g., Gitx.y.z-64-bit.exe).
3. Double-click the downloaded .exe file to start the installation.
4. You’ll be asked to configure some settings. You can keep most defaults
5. Click Finish when done.

4.2 Setup Python Environment

This chapter specifies steps to install Python application development environment. You can skip this chapter if you will not be using Python language for application development.

1. Go to the official Python website: https://www.python.org/downloads/windows/

2. Download Windows installer (64-bit) or Download Windows installer (32-bit) as per your PC configuration
3. Double-click the downloaded file: python-x.x.x-XXXX.exe
4. Check the box at the bottom that says: ☑ Add Python 3.x to PATH. This makes Python accessible from the Command Prompt.
5. You have two choices for the installation
6. Install Now (recommended for beginners) Installs Python with default settings.
7. Customize Installation
   Let's you choose features, install location, etc.
   (Only needed for advanced users.)
8. Click “Install Now”.
9. Wait for Installation to Complete
10. Once done, you’ll see a “Setup was successful” message.
11. Click Close.

4.3 Set up Java Environment

This chapter specifies steps to install Java application development environment. You can skip this chapter if you will not be using Java SDK and language for application development.

Step 1: Download Java JDK

- Go to the official Oracle JDK download page: https://www.oracle.com/java/technologies/javase-downloads.html
- Click on the latest JDK download link (e.g., "JDK 21" or "JDK 17").
- Under Windows, download the x64 Installer .exe file (e.g., jdk21_windows-x64_bin.exe).

Step 2: Run the Installer

- Double-click the .exe file you downloaded.
- Click Next to begin the installation.
- You can leave the default installation path (e.g., C:\Program Files\Java\jdk-21) or choose a custom one.
- Click Next - Wait for installation to finish - Click Close.

Step 3: Set JAVA_HOME and Update PATH (Optional but Recommended

- Open Environment Variables:
  - Press Windows + S, search for “Environment Variables”, and open:
    - Edit the system environment variables > Environment Variables
- Set JAVA_HOME:
  - Click New under *System Variables*.
  - Variable name: JAVA_HOME
  - Variable value: path to your JDK installation (e.g., C:\Program Files\Java\jdk-21)
- Update PATH: o In System Variables, find the Path variable and click Edit.
  - Click New and add:
  - %JAVA_HOME%\bin
  - Click OK to close all dialogs.

4.4 Setup C++ Environment

4.4.1 Visual Studio Code

This chapter specifies steps to download and install Visual Studio Code for C++ application development.

1. Open a web browser and go to the Visual Studio Code official website: https://code.visualstudio.com/
2. Click Download for Windows to download the Visual Studio Code installer.
3. After the download is complete, open the installer file VSCodeUserSetup.exe.
4. Follow the installation wizard:
   - Accept the license agreement. o Choose the installation location

     (default location is recommended).
   - Select additional tasks such as Add to PATH and Create Desktop Icon.
5. Click Install to begin the installation.
6. Once the installation is complete, click Finish to launch Visual Studio Code.

4.4.2 Microsoft Visual Studio

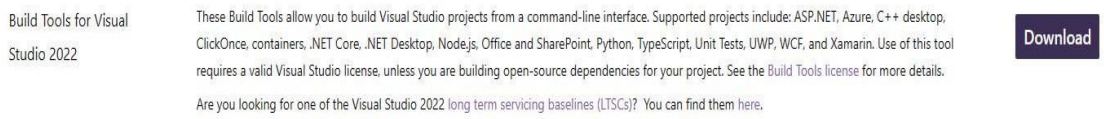
This chapter specifies steps to install C++ application development environment with Microsoft Visual Studio. You can skip this chapter if you will not be using C++ language for application development.

1. If you have a recent version of Visual Studio, open the Visual Studio Installer from the Windows Start menu and verify that the C++ workload is checked. If it's not installed, then check the box and select the Modify button in the installer

2. You can also install the Desktop development with C++ workload without a full Visual Studio IDE installation. From the Visual Studio Downloads page, scroll down until you see Tools for Visual Studio under the All-Downloads section and select the download for Build Tools for Visual Studio 2022.

   - *Here is the download link for the Build Tools for Visual Studio 2022*:

     https://aka.ms/vs/17/release/vs_BuildTools.exe



3. This will launch the Visual Studio Installer, which will bring up a dialog showing the available Visual Studio Build Tools workloads. Check the Desktop development with C++ workload and select Install

4.4.3 MSYS2

This section describes the steps to install the MSYS2-based C++ development environment, including g++ and Ninja. This setup is required to support the build process and ensure compatibility with the project requirements (C++23 or later).

1. Install Ninja generator and g++ compiler from MSYS2. This will ensure that C++ version is 23 or greater.

   I. To install g++ and Ninja download msys2 from https://www.msys2.org and run the installer and follow the instructions

   II. Next open MSYS2 MSYS from the Start Menu

   III. Run this command in MSYS2 shell:

- pacman -Syu

IV. Close the shell when prompted then reopen it and run again:

- pacman -Su

V. Next run:

- pacman -S mingw-w64-x86_64-gcc mingw-w64x86_64-ninja

VI. Add the following path to your System Environment Variables or User Environment Variables PATH (either option will work):

- C:\msys64\mingw64\bin

4.5 Install CMake

Step 1: Download CMake

- Open your browser and go to https://cmake.org/download/.
- Download the Windows installer file (.msi) from the “Latest Release” section.

Step 2: Install CMake

- Double-click the downloaded .msi file to start the installation.
- Follow the installation wizard by clicking “Next.”
- When you see the option “Add CMake to the system PATH for all users” or “Add CMake to the PATH”, make sure to select it.
- Continue with the installation by clicking “Install.”
- Once installation is complete, click “Finish.”

Step 3: Verify CMake in PATH Using Visual Studio Code (VS Code)

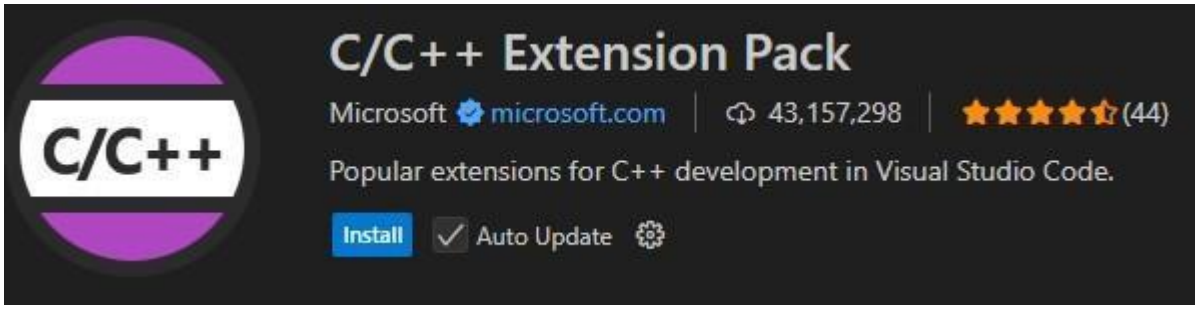
- Open Visual Studio Code
- Open the built-in terminal:
  - Go to the top menu: View - Terminal
  - or use the keyboard shortcut: Ctrl + (Ctrl and backtick key)
- In the terminal, type the following command: cmake --version

- Press Enter.
- If you see output like:
    - cmake version 3.29.2 (your version number may vary)

5 Setting up your coding environment

Each of the following steps is crucial when setting up your coding environment and using the library and cannot be skipped. Using C++23 or C++ version 23 is a requirement of this project.

1. Download the Graphiti API to anywhere on your computer that is under your user. It is available at:
2. Now open the library in visual studio code by selecting “File” in Visual Studio Code’s activity bar. Now select “Open Folder”. Then select the place that you installed the library folder
3. Next, install the “C/C++ Extension Pack” by Microsoft



4. Next, install the powershell extension “Powershell” by Microsoft

5. Now, open the powershell extension by selecting “Terminal” in Visual Studio Code’s activity bar. Then select “New Terminal”. This should open a powershell window under the terminal page on the bottom half of your screen in Visual Studio Code.
6. Next let’s ensure you can run a ps1 script file. In the library there is an exe file

that will allow the current user to execute ps1 script files.

I. Using the VS Code Explorer tab, which can be opened on the left with Ctrl + Shift + E, open the scripts folder.

II. Next right click on the file **Set-ExecutionPolicy.exe** and choose the option "Reveal in File Explorer" or alternatively left click the file then press Shift + Alt + R

III. Next, in the File Explorer right click **Set-ExecutionPolicy.exe** and choose the option "Run as administrator"

IV. It will prompt you to give it permission to run, when this happens select yes.

V. Then a message box will appear that says, "Execution Policy set to RemoteSigned", press okay or close the window to exit.

VI. If you wish to read the contents of this exe file, it is in the Set-ExecutionPolicy.ps1 file that the exe is made from.

7. With the powershell window open in Visual Studio Code or "VS Code", install the library and it's dependencies by running the script file called "install_library_vcpkg.ps1". To do this enter the command in powershell:

```
.\scripts\install_library_vcpkg.ps1
```

Note: If you encounter a PowerShell security or execution policy error while running the script, run the following command in the PowerShell window:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy
Bypass
```

If the command does not execute, ensure you have valid permissions. If the

window prompts us to give the file permission to execute, then do so. If you do not have valid permissions, run this command in an administrator terminal by opening a terminal as administrator, then navigating to the directory you installed the library, then running the same command.

8. If you wish to install the library using a specific generator and compiler do these steps. Navigate to the lib folder from the root directory by doing:

```
cd lib
```

If vcpkg is installed under your user profile you can compile the library using the library.ps1 script file. This file allows you to specify the generator and/or compiler using parameters like so:

```
.\library.ps1 -Generator "Ninja" -Compiler "g++"
```

Or you can specify no parameters by doing:

```
.\library.ps1
```

This will use your default generator and compiler.

9. Now add the include paths of your installations to the Include path of your C/C++ Configurations tool. You can open this any time with Ctrl+Shift+P .
10. If you are using JAVA for the application development, then add the below mentioned include path

    Add these to you include path with Ctrl+Shift+P (C\C++ edit configuration)

```
${vcpkgRoot}/x64-windows/include

C:\Program Files\Java\jdk-20\include

C:\Program Files\Java\jdk-20\include\win32

${env:USERPROFILE}\graphiti\include
```

11. Now your environment is all set up. If you wish to test the CPP library the run the tests using:

```
.\scripts\tools.ps1 standard_debug
```

12. The wrappers can be used as stated in their respective README.md files

    README.md file for C wrapper: \Graphiti-API-Library\binding\C

    README.md file for Python wrapper: \Graphiti-API-Library\binding\python

    README.md file for Java wrapper: \Graphiti-API-Library\binding\\jgraphiti

    1. The project includes bindings for Graphiti & Graphiti plus device in both Java (JNI) and Python. Detailed run steps are provided in the respective folders' README.md files.
    2. Java Binding (JNI) - Folder: `binding/jgraphiti` See full run instructions in: `binding/jgraphiti/README.md`
    3. Python Binding - Folder: `binding/python` See full run instructions in: `binding/python/README.m

6 Writing a CPP project

1. Start with making a CMakeLists.txt file with the contents below:

```
 cmake_minimum_required(VERSION 3.15)

set(CMAKE_CXX_STANDARD 23)

add_definitions(-D_WIN32_WINNT=0x0601)

# Set vcpkg toolchain (adjust path as needed)

# Make sure vcpkg has asio

set(CMAKE_TOOLCHAIN_FILE
"$ENV{USERPROFILE}/vcpkg/scripts/buildsystems/vcpkg.cmake")

# Set path for Graphit library

set(CMAKE_PREFIX_PATH "$ENV{USERPROFILE}/graphiti")



# Your project

project(RunGraphiti)



# Find asio for Graphiti

find_package(asio REQUIRED)

# Find Graphiti

find_package(Graphiti REQUIRED)

# Create Executable add_executable(main src/main.cpp)
# Link Executable to Graphiti Library
target_link_libraries(main PRIVATE
Graphiti::Graphiti)
```

2. To include the graphiti library do
   `#include <Graphiti/Extension.hpp>`

   This includes all the functions of the Graphiti API you'll need for this project

3. Next call startUpVCP with your COM port identified in step four of “Setting up the Graphiti”, followed by whether you wish to enable or disable key and touch events using bool values. For example, startUpVCP(“COM4”, false, false) will disable both key and touch events and use the COM port COM4
4. Next run any code you wish interacting with the graphiti api by doing
5. “graphiti->function” where “function” is any function call you wish.
6. Finally call shutDownVCP with the same boolean values followed by delete graphiti to delete the graphiti object.
7. Generally between most calls to the graphiti a graphiti->sleep(2) is called or just a time sleep for two seconds in any way to give time for the Graphiti to send a response then checking the response.
8. Example code is given below

```
#include <cstdio>
#include <Graphiti/Extension.hpp>

GraphitiExtension* graphiti = new GraphitiExtension();
void sleepAndOutput(){      graphiti->sleep(2);
auto output = graphiti->getNextOutputEvent();
```

```
     if(output.has_value()) {          std::cout
<< output.value() << std::endl;
    } else {          std::cout << "No value"
<< std::endl;
    }
}  int main() {      bool
keyEventsBool = false;     bool
touchEventsBool = false;
     std::cout << "Starting" <<
std::endl;     graphiti = new
GraphitiExtension();
    if(!graphiti->startUpVCP("COM4",
keyEventsBool, touchEventsBool)){         return
1;
    }     graphiti->getDateAndTime();
sleepAndOutput();     graphiti-
>shutDownVCP(keyEventsBool, touchEventsBool);     delete
graphiti;     std::cout << "Ending" << std::endl;
return 0;
}
```